FELIX

An Emerging Autonomous Engineering Control Plane

What it is, what the measured workflow shows, and a qualitative assessment of why the setup is unusually ambitious.

Bottom line: Felix is more interesting than the raw Claude usage. The usage is extreme; Felix is the attempt to turn that extreme autonomy into a governed engineering system - separating capability from authority, model judgment from mechanical evidence, and execution from trust.

87,137	62,266	~109K	18.1B	$16.6K
unique API model turns	nested subagent turns	tool calls	reported token events	ccusage API-equivalent cost

Why this report exists. Earlier analysis looked at a very high-volume Claude Code workflow and saw an unusually autonomous agent system. Once Felix and its design history were introduced, the interpretation changed: much of the recent activity is not merely high-volume coding assistance; it is running under a developing governance layer whose stated endgame is an autonomous engineering control plane.

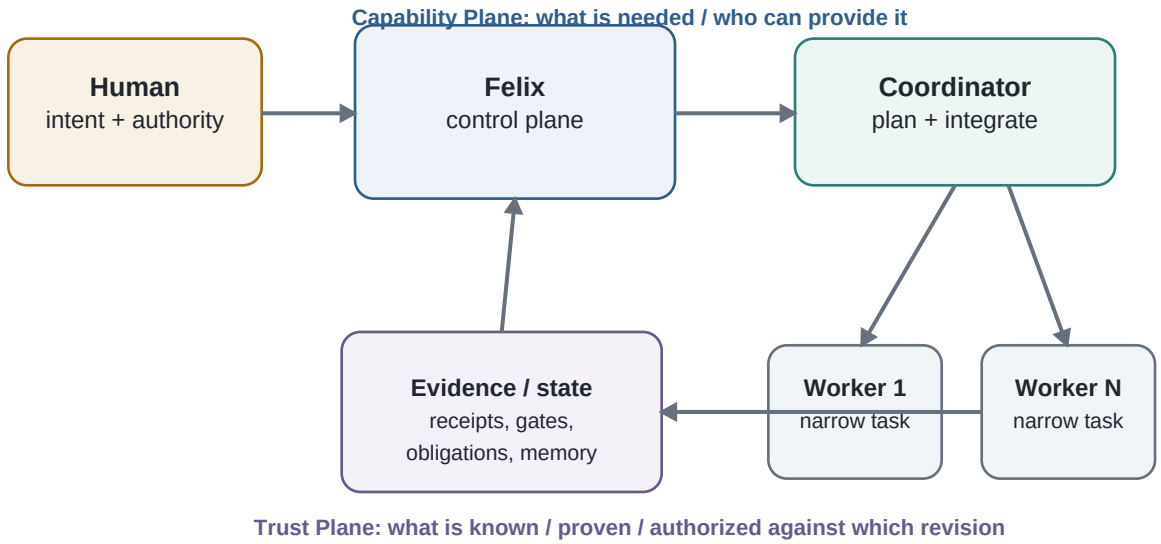
What is observed vs. intended. Quantitative usage figures in this report come from the local Claude transcript analysis and ccusage totals supplied in this conversation. Architectural descriptions come from the supplied Felix design documents. Several mechanisms are explicitly pending, partial, or design-only; those are labeled as such.

Executive take

Observed	Delegation is already dominant. Subagents are ~71.5% of unique model turns overall and ~79.4% in the commissioned-Felix phase. They produce most output while carrying far smaller contexts than the coordinator.
Measured shift	From pre-Felix to commissioned Felix, turns per human prompt rose ~34%, tools/prompt ~57%, output/prompt ~60%, and context/output improved ~28%; API-equivalent cost/prompt rose ~24%.
Architectural thesis	Felix is explicitly framed as a control plane with a Capability Plane and Trust Plane, returning to the human when judgment or authority is genuinely irreducible.
Assessment	Impressive and unusually ambitious; still immature. The strongest idea is not more agents - it is institutionalizing evidence, authority, lifecycle, and escalation around agents already doing substantial work.

1. What Felix is trying to become

The newest supplied direction memo states the endgame directly: Felix is an **autonomous engineering control plane**, not merely a better coding agent or a trust kernel. The intended division is that the human supplies the idea, taste, and important/irreducible choices, while Felix determines what engineering must exist, assembles/operates it, proves it, and manages capability and trust state.



The key distinction

Capability Plane: what the project needs, which provider/worker can supply it, how well it performed, and when it should be retired. **Trust Plane:** facts, unknowns, obligations, evidence, the revision against which something was proven, and release/authority state.

> **This is the conceptual move that makes Felix more than a conventional agent harness: it is trying to govern the engineering institution around autonomous workers, not simply orchestrate tool calls.**

Important design signals

Evidence over narration	Model claims are separated from revision-bound evidence and receipts.
Capability != authority	A session may act without having authority to make every resulting state effective.
Trusted promotion	A candidate engine is verified, checked by the trusted base for authority changes, merged, then promoted.
Refutable governance	Weak advisory/routing mechanisms were measured and redesigned instead of defended.
Bounded workers	The pending lease design uses narrow headless sessions with constrained tooling and authority.

Status caveat: the September 17 distance audit explicitly marked situation triggers and a stopping rule as absent, and release/evidence/provider lifecycle pieces as partial. The September 19 direction document also contains future control-plane/lease designs. The architecture is ahead of full implementation.

2. Quantitative footprint of the observed workflow

The full aligned analysis covers 73 active days (22 Jun - 21 Sep 2026). The scale is unusual even before asking whether it is efficient.

2,679	24,871	62,266	89.6M	~450K
estimated human prompts	top-level turns	subagent turns	output tokens	main avg context / turn

Metric	Top-level / coordinator	Nested subagents	Interpretation
Unique API turns	24,871 (28.5%)	62,266 (71.5%)	Most reasoning calls are delegated.
Context processed	11.208B (62.2%)	6.816B (37.8%)	Coordinator carries most state despite fewer turns.
Output generated	30.1M (33.6%)	59.5M (66.4%)	Workers produce about two-thirds of output.
Avg context / turn	450,644	109,472	Coordinator is ~4.1x larger per turn.
Context / output	372.1x	114.6x	Workers are much leaner by token-throughput ratio.

What this says: the system is already organized like a coordinator plus a large pool of narrower workers. The raw scale is extreme; the more interesting question is whether Felix makes that scale safer, more productive, and less dependent on human attention.

Coordinator-context nuance

The earlier temptation was to call ~450K average coordinator context “bloat.” Later transcript analysis weakened that conclusion. Late-session tool errors were reported lower than early-session errors; repeated file reads were near zero; human correction rates did not rise; and observed handoff failures were more concrete than evidence of long-context degradation. The better framing is that coordinator context is **valuable state with a carrying cost**, not automatically a defect.

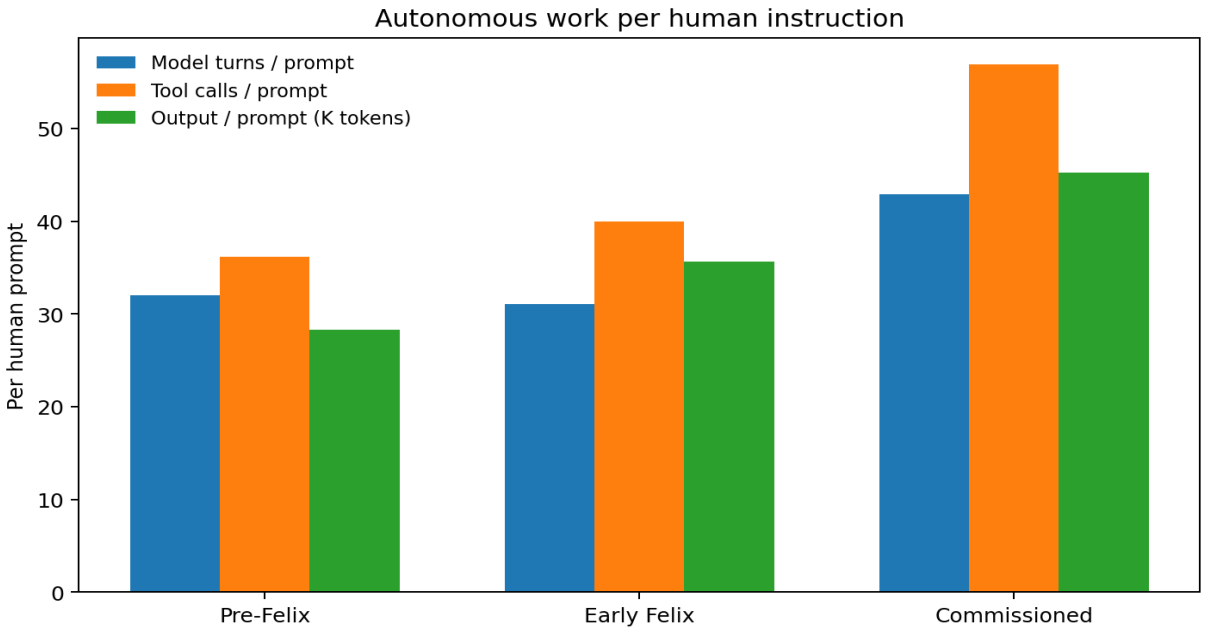
Cost note

The reported $16,641 figure is ccusage/API-equivalent cost, not necessarily money actually paid on a subscription. Most coordinator input is cache-read traffic, so raw context-token reductions do not translate one-for-one into dollar savings.

3. What changed across the Felix phases

A three-phase split is more faithful than a simple before/after: pre-Felix, early/prototype Felix beginning with ledger activity on 11 Aug, and a commissioned regime beginning around 4 Sep as projects were formally commissioned. This is observational, not a randomized causal experiment.

Phase	Days	Prompts	Turns	Subagent share	Main avg ctx	Turns/prompt	Tools/prompt	Output/prompt	Ctx/out	Cost/prompt
Pre-Felix	36	872	27,914	69.7%	462K	32.0	36.2	28.3K	244.5	$5.94
Early Felix	21	913	28,349	64.6%	450K	31.1	40.0	35.6K	192.0	$6.70
Commissioned	**16**	**724**	**31,072**	**79.4%**	**436K**	**42.9**	**56.9**	**45.2K**	**177.0**	**$7.39**

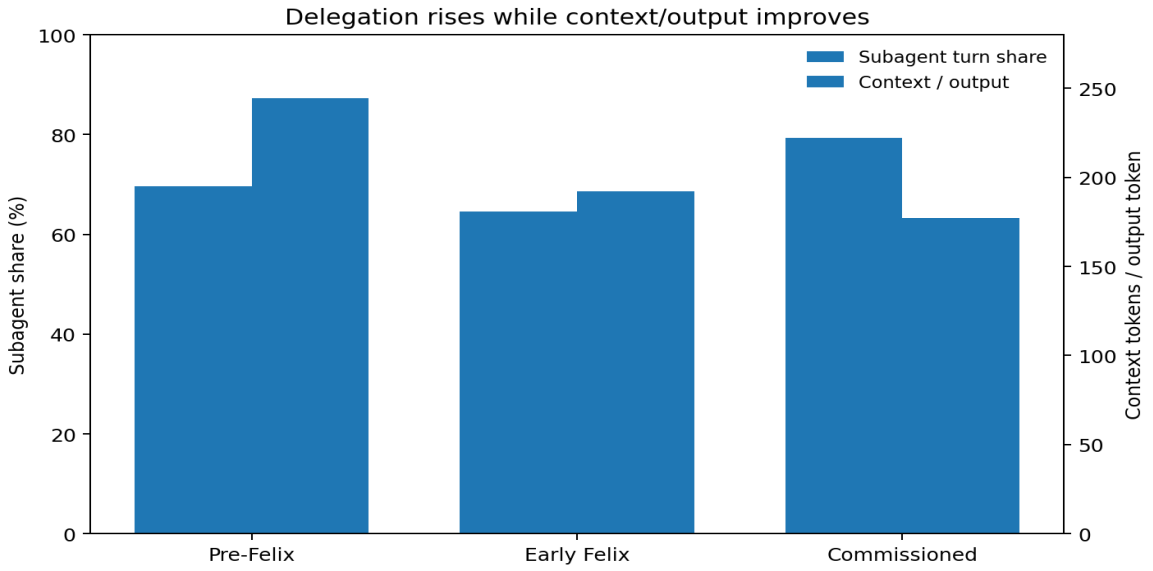


Pre-Felix -> commissioned Felix

Subagent turn share	69.7% -> 79.4%	+9.7 percentage points
Model turns / human prompt	32.0 -> 42.9	+34%
Tool calls / human prompt	36.2 -> 56.9	+57%
Bash calls / human prompt	22.5 -> 43.0	+91%
Output / human prompt	28.3K -> 45.2K	+60%
Context / output	244.5 -> 177.0	-27.6% (better throughput)
API-equivalent cost / prompt	$5.94 -> $7.39	+24%

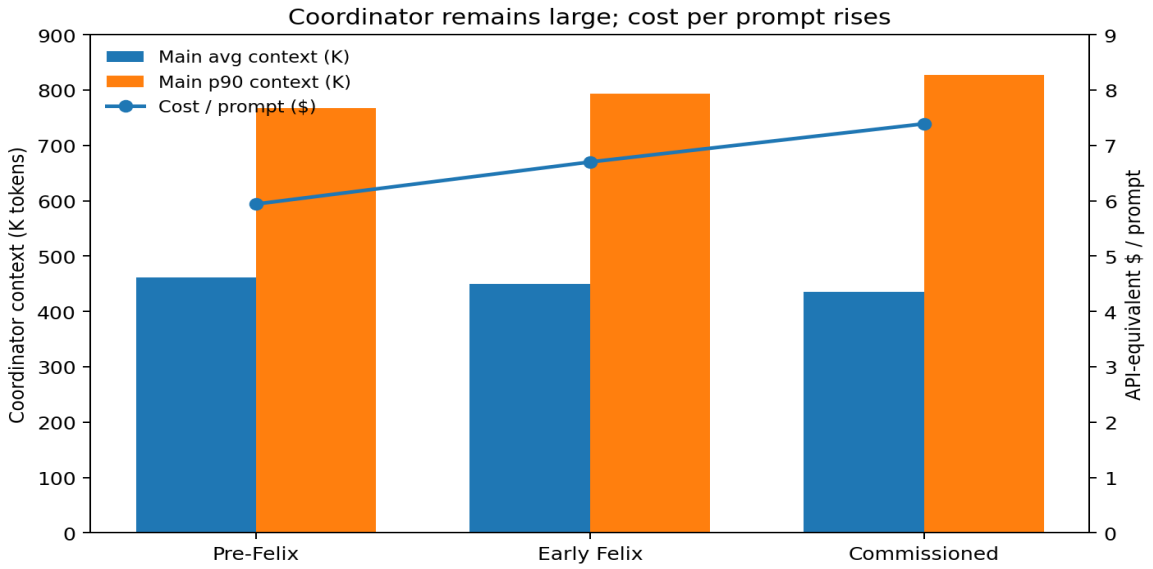
Interpretation: commissioned Felix coincides with markedly more work per human instruction and substantially heavier delegation. It also coincides with improved context/output efficiency, but not lower absolute compute or lower cost per prompt. A reasonable reading is “more autonomous engineering depth purchased with more compute,” not “Felix makes Claude cheaper.”

4. The result I find most interesting



The notable combination: the commissioned phase reaches 79.4% subagent turns while context/output falls to 177 from 244.5 pre-Felix. At the same time, output per human prompt rises to 45.2K tokens. This is consistent with an architecture that pushes more concrete work into narrower worker contexts while the coordinator remains the integration layer.

What Felix has not solved yet



Coordinator average context falls only modestly (462K -> 436K), while p90 rises (767K -> 827K). So commissioned Felix does **not** yet look like a context-lifecycle solution. The later analysis suggests that may be fine: long-lived context is not demonstrably harming quality. The higher-confidence problem is unmanaged lifecycle - stale long-idle resumptions, stale handoff state, and sessions continuing across changes to the governing Felix engine.

5. Why the design is unusually interesting

It treats engineering trust as a first-class system.

The exception design explicitly prevents a session from making its own grant effective merely by writing it. Effective authority is read from a separately controlled/base state. This is closer to a policy/authority system than to a prompt guardrail.

It thinks about self-governance.

The promotion design distinguishes candidate verification from authorization by the currently trusted evaluator, then promotes the new evaluator only after merge. A system upgrading the governor that governs itself is a genuinely hard control-plane problem.

It is willing to kill weak agentic ideas.

An earlier advisory/routing mechanism measured roughly 2% precision in a blind study; the redesign explicitly concluded that low compliance was often correct non-compliance. Later direction work also reversed a typed-runtime recommendation after evidence did not support it.

It is moving toward bounded autonomous workers.

The pending control-plane v0 lease design describes headless, restricted sessions with narrowly selected tools/permissions and an explicit result/hold record. That is a much more disciplined unit of autonomy than “spawn another full agent.”

It optimizes human authority, not merely model throughput.

The advisory redesign keeps founder interruptions per session as an objective but pairs it with missed escalations so the metric cannot be gamed by simply advising less.

> **My qualitative reaction: the impressive part is not “lots of agents.” It is that the builder has generated enough real autonomous activity to encounter institutional problems - authority, evidence, promotion, exceptions, lifecycle, escalation - and is trying to encode answers as infrastructure rather than relying on the model to remember good behavior.**

6. Why I also call it extreme

Scale	~87K unique model turns, ~62K nested subagent turns, ~109K tool calls, ~80K Bash calls, ~18.1B reported token events in 73 active days.
Autonomy	Commissioned phase: ~43 model turns and ~57 tool calls per estimated human prompt, with nearly 80% of model turns delegated.
State size	Coordinator calls average roughly 450K context tokens overall; several high-volume sessions repeatedly operated near the ~1M working-context region.
Self-reference	Felix is used while Felix itself is being redesigned, promoted, and governed - creating stale-engine, promotion, and authority questions most coding assistants never face.
Control-plane ambition	The stated loop spans intent, required engineering, obligations/unknowns, capabilities, discovery/admission, evidence, release, telemetry, reconciliation, and retirement.

What keeps this from being pure hype

The supplied documents contain explicit negative findings and incompleteness: weak advisory precision, partially built evidence/release contracts, absent stopping rules/situation triggers at the time of the audit, handoff failures observed in practice, and design reversals when measurements failed to support an idea. That makes the project more credible to me than a document set in which every subsystem is described as solved.

The danger

This level of autonomy has a large execution surface. A governance bug can become systemic because it affects many downstream actions. Shell access, stale state, mistaken authority, and incorrectly trusted evidence can propagate quickly. Felix therefore needs conventional hard barriers underneath it - Git/CI, branch or authority protections where available, deterministic verification, secret/production isolation - rather than treating the control plane as magic safety.

> **Current characterization: a powerful, somewhat unruly autonomous engineering system that is gradually turning its recurring failure modes into explicit control-plane machinery.**

7. Current assessment and what would change my mind

Very impressive	The architecture targets second-order autonomy problems rather than merely adding more agent loops.
Very extreme	The observed execution volume is closer to a small software organization worth of machine activity than ordinary pair-programming use.
Not obviously wasteful	Most delegation is comparatively lean; the major compute burden is the long-lived coordinator, whose continuity appears to have real value.
Not mature yet	Some core control-plane pieces were still absent/partial in the latest supplied gap audit, and transition/handoff behavior has produced real failures.
Promising, not proven	The pre/early/commissioned phase data is observational and confounded by project/model/workload changes. It supports hypotheses, not a causal claim that Felix created every improvement.

Highest-value next evidence

The next convincing step is not a prettier architecture diagram. It is showing that Felix improves **verified engineering progress per unit of human attention and compute**. Useful measurements include completed/verified milestones per human intervention, missed escalations, handoff defects, stale-governance incidents, rework/error rates, lifecycle outcomes (continue vs compact vs restart), and cost per verified milestone.

Near-term lifecycle lesson

Do not reduce coordinator context simply because it is large. Treat continuity as a governed resource. Restart after governing-engine promotion; harden verified handoffs; avoid expensive long-idle resumptions when continuity/caching has gone stale; and fold context/lifecycle decisions into Felix's planned situation-trigger/stopping-rule architecture rather than bolting on a separate token optimizer.

> **If the project works: Felix would not merely make Claude code faster. It would make it possible for a human to remain at the level of intent, taste, and irreducible authority while an increasingly autonomous software organization handles much of the engineering execution and proof underneath.**

Sources and methodology

- Local Claude Code transcript analysis and ccusage totals supplied in this conversation (73 active days; globally deduplicated model-message accounting reconciled to ccusage within ~0.1%).

- 2026-09-19-direction-decisions.md: adopted control-plane endgame; Capability/Trust planes; hook direction; control-plane v0 lease design.

- 2026-09-17-v32-distance.md: measured built/partial/absent v3.2 mechanisms, including absent situation triggers/stopping rule at that snapshot.

- 2026-09-17-exception-channel-design.md: authority separation for task-blocking exceptions.

- 2026-08-18-promotion-spec.md: trusted evaluator promotion lifecycle and authority-kernel reasoning.

- 2026-08-30-advisory-redesign.md: measured advisory failures, redesign, per-session drill, and founder-interruption metric correction.

- 2026-09-18-ecc-against-v32.md and 2026-09-16-discovery-baseline-design.md: capability-provider/discovery context and outstanding gaps.

Limitation: several documents are dated designs/snapshots and may be stale relative to the live repository; proposed mechanisms are not assumed deployed.